

HEPDataset: A configurable framework for constructing machine-learning datasets from Delphes ROOT files

Yusheng Wu¹ and A. M. M Elsayed¹

¹ Department of Modern Physics and State Key Laboratory of Particle Detection and Electronics, University of Science and Technology of China, Hefei, China  Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Searches for physics beyond the Standard Model (BSM) at the LHC increasingly rely on multivariate classifiers — boosted decision trees (BDTs) or neural networks — trained on flat, per-event feature tables built from reconstructed-object kinematics, rather than on a small number of hand-picked cut variables. Building these tables from Delphes (Favereau et al., 2014) fast-detector-simulation output is, however, a recurring and largely bespoke engineering task: every analysis re-implements object quality selections, signal-region classification, N-body kinematic combinatorics, and cross-section-weighted sample bookkeeping from scratch.

HEPDataset is a Python package that generalizes this task into a configurable pipeline. Users supply (i) a registry of background and signal Delphes ROOT samples with associated cross sections, defined via YAML; (ii) object definitions specifying which reconstructed objects and kinematic branches to read, including automatically generated combinatorial N-body observables (ΔR , $\Delta\phi$, M_T , M_{T2} , and others) for user-chosen object groupings; and (iii) a signal-region classification function that maps per-event reconstructed objects to a signal-region label. The package reads events via ROOT's TTreeReader/ExRootTreeReader interfaces (Brun & Rademakers, 1997), applies the user's object and region logic, computes event-shape and kinematic observables (some accelerated via numba (Lam et al., 2015)), and writes the result to per-region tabular output (ROOT trees, Parquet, or CSV) suitable for direct use with tools such as XGBoost (Chen & Guestrin, 2016). Event processing is parallelized across chunks of each sample, with per-chunk outputs merged via hadd, allowing datasets built from many signal mass points and background processes to scale across available cores without manual chunk management.

Machine learning has become an essential component of modern high-energy physics (HEP), with applications ranging from event classification and anomaly detection to particle reconstruction and jet tagging. Despite the availability of mature software for event generation, detector simulation, and data analysis, there remains no lightweight framework dedicated to transforming simulated Delphes events into machine-learning-ready tabular datasets.

HEPDataset is an open-source Python framework designed to bridge this gap. The package reads Delphes ROOT files, performs configurable object selection, computes user-defined physics observables, constructs multi-object kinematic variables, and exports flattened datasets in formats suitable for downstream machine-learning workflows.

The framework is analysis-independent and is configured through simple YAML files together with optional user-defined Python analysis modules. This allows users to describe physics objects, define signal regions, specify feature sets, and process arbitrary collections of signal and background samples without modifying the core software.

Statement of need

Several mature, public frameworks exist for confronting BSM models with LHC data using Delphes-level simulation: MadAnalysis 5 (Conte et al., 2013; Conte & Fuks, 2018), CheckMATE (Dercks et al., 2017; Drees et al., 2015), and Rivet (Bierlich & others, 2020). These tools excel at *recasting* — reproducing a specific, already-published experimental analysis (its cuts, signal regions, and efficiencies) so that a new theoretical model can be tested against it. Their unit of output is typically a small number of signal-region yields or efficiencies, matched to a specific published cut-and-count or simplified-likelihood analysis.

This is a different problem from the one facing an analysis that is *itself* being designed around a multivariate classifier, before any public recast-ready implementation exists. In that setting, what is needed is not a yield in a fixed set of signal regions, but a large, flat table of per-event kinematic features — spanning single-object kinematics, all relevant N-body combinations, and event-shape variables — computed consistently across many background processes and many signal mass points, correctly weighted by cross section and luminosity, and split by an analysis-specific signal-region definition that may itself evolve during BDT development. Assembling this by hand for every new analysis leads to substantial duplicated engineering effort across phenomenology groups, and to pipelines whose object/branch/region logic is difficult to disentangle from I/O and parallelization concerns, making them hard to validate or reuse.

HEPDataset targets this gap directly: it separates *what to read* (object and kinematic definitions), *how to select* (per-object quality selections and a user-supplied signal-region function), and *what samples to run over* (a YAML-configured, cross-section-weighted sample registry) from the I/O, chunking, and merging machinery, which is handled once, generically, for any configuration. The result is a reusable tool for producing ML-ready datasets from Delphes samples that complements, rather than duplicates, existing recasting frameworks: it is intended for the dataset-construction stage of a BDT- or NN-based search, upstream of tools such as pyhf (Heinrich et al., 2021) that are used for the subsequent statistical inference.

Preparing machine-learning datasets is one of the most repetitive tasks in collider phenomenology. Although Delphes provides detector-level ROOT files and ROOT itself offers flexible event access, each analysis typically develops a custom event loop that performs object selection, computes derived kinematic variables, flattens event information, and writes a dataset for machine learning.

These analysis-specific implementations often duplicate substantial amounts of code, making them difficult to reuse, maintain, or compare across different analyses. Furthermore, introducing additional reconstructed objects or derived features usually requires modifying multiple components of the analysis pipeline.

HEPDataset addresses this problem by providing a configurable feature engineering framework for Delphes events. Rather than hard-coding variables inside the event loop, users specify

- physics objects,
- object representations,
- analysis selections,
- signal-region definitions,
- desired observables,
- output formats,

which are automatically translated into a complete event-processing pipeline.

State of the field

A description of how this software compares to other commonly-used packages in the research area. If related tools exist, provide a clear “build vs. contribute” justification explaining your unique scholarly contribution and why existing alternatives are insufficient.

Software design

An explanation of the trade-offs you weighed, the design/architecture you chose, and why it matters for your research application. This should demonstrate meaningful design thinking beyond a superficial code structure description.

Acknowledgements

The author acknowledges the support of the University of Science and Technology of China and valuable discussions with members of the particle physics group.

AI usage disclosure

Transparent disclosure of any use of generative AI in the software creation, documentation, or paper authoring. If no AI tools were used, state this explicitly. If AI tools were used, describe how they were used and how the quality and correctness of AI-generated content was verified.

References

- Bierlich, C., & others. (2020). Robust independent validation of experiment and theory: Rivet version 3. *SciPost Physics*, 8, 026. <https://doi.org/10.21468/SciPostPhys.8.2.026>
- Brun, R., & Rademakers, F. (1997). ROOT — an object oriented data analysis framework. *Nuclear Instruments and Methods in Physics Research Section A*, 389(1-2), 81–86. [https://doi.org/10.1016/S0168-9002\(97\)00048-X](https://doi.org/10.1016/S0168-9002(97)00048-X)
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Conte, E., & Fuks, B. (2018). Confronting new physics theories to LHC data with MADANALYSIS 5. *International Journal of Modern Physics A*, 33(28), 1830027. <https://doi.org/10.1142/S0217751X18300272>
- Conte, E., Fuks, B., & Serret, G. (2013). MadAnalysis 5, a user-friendly framework for collider phenomenology. *Computer Physics Communications*, 184, 222–256. <https://doi.org/10.1016/j.cpc.2012.09.009>
- Dercks, D., Desai, N., Kim, J. S., Rolbiecki, K., Tattersall, J., & Weber, T. (2017). CheckMATE 2: From the model to the limit. *Computer Physics Communications*, 221, 383–418. <https://doi.org/10.1016/j.cpc.2017.08.021>
- Drees, M., Dreiner, H., Schmeier, D., Tattersall, J., & Kim, J. S. (2015). CheckMATE: Confronting your favourite new physics model with LHC data. *Computer Physics Communications*, 187, 227–265. <https://doi.org/10.1016/j.cpc.2014.10.018>
- Favereau, J. de, Delaere, C., Demin, P., Giammanco, A., Lemaître, V., Mertens, A., & Selvaggi, M. (2014). DELPHES 3, a modular framework for fast simulation of a generic

- 125 collider experiment. *Journal of High Energy Physics*, 2014(2), 57. [https://doi.org/10.](https://doi.org/10.1007/JHEP02(2014)057)
126 [1007/JHEP02\(2014\)057](https://doi.org/10.1007/JHEP02(2014)057)
- 127 Heinrich, L., Feickert, M., Stark, G., & Cranmer, K. (2021). Pyhf: Pure-python implementation
128 of HistFactory statistical models. *Journal of Open Source Software*, 6(58), 2823. [https:](https://doi.org/10.21105/joss.02823)
129 [//doi.org/10.21105/joss.02823](https://doi.org/10.21105/joss.02823)
- 130 Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based python JIT compiler.
131 *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6.
132 <https://doi.org/10.1145/2833157.2833162>

DRAFT